Frida hook

前置知识

这个板子在写什么：

正题

ISCC Puzzle_Game

先看外面包裹的一层

里面一层

ISCC Challenge_Mobile

~~本人的一些描述很不准确，而且会经常用一些大白话和一些不专业的词汇，请见笑~~

~~特别是比较喜欢用cpp的描述，java的描述太拗口了，听懂就好~~

前置知识

环境什么的就不说了，老生常谈了

常用指令 Plain Text

```
adb shell
su
cd data/local/tmp
./frida-server

adb forward tcp:27042 tcp:27042
frida-ps -Ua
```

然后就是大家常看见的板子

板子 Plain Text

```
import frida
import sys
jscode = '''

'''


def printMessage(message,data):
    if message['type'] == 'send':
        print('[*] {0}'.format(message['payload']))
    else:
        print(message)

process = frida.get_remote_device().attach(4083)
script = process.create_script(jscode)
script.on('message',printMessage)
script.load()
sys.stdin.read()
```

因为本人的技术实在太水，js和java代码技术烂的一坨,解读这些还是花了一段时间

这个板子在写什么：

1.首先是jscode，就是核心部分，要用的代码块

2.在frida里面，js代码可以通过send函数发消息给python端，大概的长这个样子：

Plain Text

```
send({ type: 'info', payload: 'Hello from JavaScript' });

```

printMessage这个函数的意思就是说，接收一个字典message作为参数，，首先看message的键值对再进行处理

3.首先remote连接了一个安卓的进程，创建了一个js的脚本（15行），然后加载到进程中（17行）

4.设置了一个监听器，用来实现python和js之间的互动（只要js用send函数，python就会收到）

这个板子的目的在于不用再手动注入，那样特别麻烦

手动注入长这个样子：

```
frida -U -l hook.js -f
```

用python这个板子，你就只需要把写好的js代码放到那个代码块里面，然后板子帮你注入

正题

ISCC Puzzle_Game

做了看了已知有个getstr方法比较关键，前面的数字可以爆，也可以逆，无非就8位，后面的字符串才是关键

可以尝试hook，先上脚本

```Plain Text
Java.perform(function() {

    var targetClass = Java.use('com.example.whathappened.MyJNI.Myjni');
    targetClass.getstr.implementation = function() {
        var ReturnValue = this.getstr();
        console.log('return value: ' + ReturnValue);
        return ReturnValue;
    };
});
```

先看外面包裹的一层

```Plain Text
Java.perform(function);
```

这个方法会默认内部调用一个VM::AttachCurrentThread，把当前线程attach到当前的Java虚拟机上，要求参数传一个函数的handle，并且会直接调用这个函数，最后调用结束之后会执行VM::DetachCurrentThread 接触attach

里面一层

```
function() {
    var targetClass = Java.use('com.example.whathappened.MyJNI.Myjni');
    targetClass.getstr.implementation = function() {
        var ReturnValue = this.getstr();
        console.log('return value: ' + ReturnValue);
        return ReturnValue;
    };
}
```

这里就是写了那个自动调用的方法的方法实现，先载入目标package，Myjni。

接着关键点

```
targetClass.getstr.implementation = function()
```

这里重新写了这个类里的getstr方法，具体怎么重写呢，起了一个匿名的函数叫function（也可叫new_getstr，反正无所谓，和外层的function不是一个东西），然后写了它的实现

```
 var ReturnValue = this.getstr();
console.log('return value: ' + ReturnValue);
  return ReturnValue;
```

这里用了个this指针，这个this指针还是指代原来的的那个，没有被修改的方法

中间打一个log，把这个值打印出来，接着打印出来

最后返回

ISCC Challenge_Mobile

写的时候也是遇到了要用hook来hook出那个不落地的dex

通过载入一个叫ming的文件，然后通过本地库smc解密一下

```
try {
    Class loadClass = (Build.VERSION.SDK_INT >= 29 ? new InMemoryDexClassLoader(new ByteBuffer[]{ByteBuffer.wrap(m99a(LoadData("ming")))}, new File(((PathCl
    Method method = loadClass.getMethod("isflag", String.class);
    if (!str.substring(0, 5).equals("ISCC{") || str.charAt(str.length() - 1) != '}') {
```

```
    TextView tv_1;

    /* renamed from: a */
    public native byte[] m99a(byte[] bArr);

    static {
        System.loadLibrary("challengemobile");
    }

    /* JADX INFO: Access modifiers changed from: protected */
```

但是这里的m99a并不是真正的名字，是因为jadx认为这个东西有混淆，改了一下名字，上面才是真正的名字

也是hook返回值

```Plain Text
Java.perform(function () {

    let MainActivity = Java.use("com.example.challengemobile.MainActivity");
    MainActivity.a.implementation = function (bArr) {
        let result = this.a(bArr);
        console.log(`MainActivity.a result=${result}`);
        return result;
    };
});
```